交互题

人员

许睿谦、田牧可 到课

上周作业检查

上上周作业链接: https://vjudge.net/contest/818166

概览 题目 提交状态 排名 (14:05:20:34) 讨论

Rank	Team	Score	Penalty	A 6 / 9	B 6 / 6	C 4 / 4	D 3 / 20	E 1 / 2	F 9 / 9
1	☆ junyan007008	5	63466	7:00:55:00	7:01:10:00	9:08:17:00	13:12:24:00 (-9)		7:00:00:58
2	☆ lyc12345bc (刘奕辰)	5	69609	7:00:57:42	7:01:28:31	13:13:26:00	13:12:59:00 (-4)	(-1)	7:01:57:59
3	☆ wcz123bc	5	85802		7:00:53:22	13:03:17:00	13:02:38:00 (-4)	13:02:24:00	13:03:30:10
4	☆ yyycj	3	30494	7:01:33:53 (-2)	7:01:47:49				7:00:13:08
5	☆ lijinshu	3	49156	13:13:00:00	7:01:12:00				13:13:04:51
6	☆ xuruiqian	3	51288	14:04:48:45		14:05:00:00			7:04:59:28
7	☆ chenxinmiao (陈欣妙)	2	20277	7:01:30:55 (-1)					7:00:06:36
8	☆ r111	2	20932		7:06:25:00				7:06:27:25
9	☆ chujinxuan (褚锦轩)	1	10150						7:01:10:48

上周作业链接: https://vjudge.net/contest/820326

本周作业

https://vjudge.net/contest/821864 (课上讲了本周的 A ~ E 题, 课后作业是本周的 A ~ E 题)

课堂表现

今天讲了 交互题 这个知识点, 今天的 E 题是今年省选真题, 相对难一点, 同学们课上都没做完, 课下要把这道题好好补一补。

课堂内容

P1733 猜数（IO交互版）

交互题练习

```cpp
#include <bits/stdc++.h>

using namespace std;

int main()
{
  int l = 1, r = 1e9;
  while (true) {
    int mid = (l + r) / 2;
    cout << mid << endl;

    int x; cin >> x;
    if (x == 0) break;
    if (x == -1) l = mid+1;
    else r = mid-1;
  }
  return 0;
}
```

CF1698D Fixed Point Guessing

比较基础的交互题, 二分找即可

```cpp
#include <bits/stdc++.h>
#define int long long

using namespace std;

const int maxn = 2e5 + 5;
int w[maxn], p[maxn];

int get_sum(int l, int r) { return p[r] - p[l-1]; }

void solve() {
  int n; cin >> n;
  for (int i = 1; i <= n; ++i) cin >> w[i], p[i] = p[i-1] + w[i];

  int l = 1, r = n;
  while (l < r) {
    int mid = (l + r) / 2;
    cout << "? " << mid-l+1 << " ";
    for (int i = l; i <= mid; ++i) cout << i << " ";
    cout << endl;

    int x; cin >> x;
    if (get_sum(l,mid) == x) l = mid+1;
    else r = mid;
  }

  cout << "! " << l << endl;
```

```
}

signed main() {
  int T; cin >> T;
  while (T -- ) solve();
  return 0;
}
```

CF1479A Searching Local Minimum

每次判断 mid 和 mid+1 位置的大小, 来决定下一次往哪边继续判断

```
#include <bits/stdc++.h>

using namespace std;

signed main() {
  int n; cin >> n;
  int l = 1, r = n;
  while (l < r) {
    int mid = (l + r) / 2;
    int a = mid, b = mid+1, fa, fb;
    cout << "? " << a << endl;
    cin >> fa;
    cout << "? " << b << endl;
    cin >> fb;
    if (fa > fb) l = mid + 1;
    else r = mid;
  }
  cout << "! " << l << endl;
  return 0;
}
```

P8430 [COI 2020] Zagrade

用括号匹配的思路来做, 每次看栈顶的两个是否匹配, 最后剩在栈里面的位置, 前一半是右括号, 后一半是左括号

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 1e5 + 5;
char s[maxn];

int main()
{
  int n, Q; cin >> n >> Q;
  stack<int> stk;
  for (int i = 1; i <= n; ++i) {
```

```
    if (stk.empty()) stk.push(i);
    else {
      int j = stk.top();
      cout << "? " << j << " " << i << endl;

      int x; cin >> x;
      if (x == 1) s[j] = '(', s[i] = ')', stk.pop();
      else stk.push(i);
    }
  }

  int len = stk.size();
  for (int i = 1; i <= len/2; ++i) {
    int x = stk.top(); s[x] = '('; stk.pop();
  }
  for (int i = 1; i <= len/2; ++i) {
    int x = stk.top(); s[x] = ')'; stk.pop();
  }

  cout << "! ";
  for (int i = 1; i <= n; ++i) cout << s[i];
  cout << endl;
  return 0;
}
```

[省选联考 2026] 排列游戏 / perm

```
#include <vector>
void init(int, int);
std::vector<int> perm(int);
int query(int, int);

#include <bits/stdc++.h>

using namespace std;

const int maxn = 3e4 + 5;
int w[maxn];

void init(int c, int t) { memset(w, 0, sizeof(w)); }

struct node {
  int value, pos;
  bool operator < (const node& p) const { return value < p.value; }
  bool operator > (const node& p) const { return value > p.value; }
};

std::vector<int> perm(int n) {
  priority_queue<node, vector<node>, less<node>> q;
  set<int> s;
  for (int i = 0; i <= n-1; ++i) s.insert(i);
```

```
  int pos_0 = 0;
  for (int i = n-2, last = n; i >= 0; i--) {
    int t = query(0, i);
    if (t < last) w[i+1] = t, last = t, s.erase(t);
    else q.push({last, i+1});

    if (t == 0) { pos_0 = i+1; break; }
  }

  for (int i = 1, last = n; i <= pos_0; ++i) {
    int t = query(i, n-1);
    if (t < last) w[i-1] = t, last = t, s.erase(t);
    else q.push({last, i-1});
  }

  if (pos_0 == 0) w[0] = 0, s.erase(0);

  while (!q.empty()) {
    node u = q.top(); q.pop();
    w[u.pos] = *s.rbegin();
    s.erase(*s.rbegin());
  }

  vector<int> vec;
  for (int i = 0; i <= n-1; ++i) vec.push_back(w[i]);
  return vec;
}
```